# 황보민

원스토어 클라이언트개발팀(9명) 기술 리드 · Android 개발자 · 조직용 AI 하네스 구축
010-4354-1115 · 1stpie@naver.com · 경기 안양시

원스토어 클라이언트개발팀의 기술 리드이자 16년 차 모바일 개발자입니다. 2026년 1월부터 조직을 위한 AI 하네스를 설계·운영해, 지금은 팀원 전원이 이 하네스로 개발하고 사람은 지시·감독·검수를 맡습니다. 이 문서는 핵심 역량마다 대표 사례를 골라 문제, 판단, 구조, 결과를 적었습니다. 경력 전체와 기술 목록은 이력서에 있습니다.




## 핵심 역량과 대표 사례

| 역량 | 대표 사례 |
|---|---|
| **조직을 위한 AX 환경 구축** | **조직용 AI 하네스**: 팀 전원이 쓰는 AI 개발 환경의 구성·루프·검증 하네스로 만든 단위 테스트와 테스트 자동화 |
| **기술 리드** | **Clean Architecture 기반 앱 구조 설계와 팀 표준**<br>제주항공 항공 예약 파트 리더 |
| **리팩토링** | **서비스 중인 국내·글로벌 앱의 전 계층 리팩토링**<br>다운로드·설치 경로의 Service에서 WorkManager 전환 |
| **모바일 개발·운영** | **다운로드 APK 무결성 검증**: 설치 시작 한 지점에서 대조<br>세션 만료 복구, 설치 비호환 단말 관리 |
| **SDK 개발** | **AAB Dynamic Delivery SDK**: SDK와 원스토어 앱 사이 규격 설계<br>원스토어용 bundletool 라이브러리, 재서명·리패키징 도구, 물리엔진 SDK |

# 1. 조직을 위한 AX 환경 구축

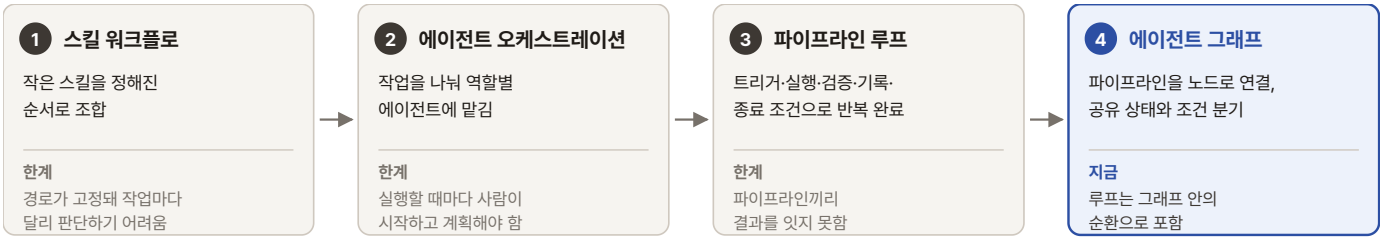
## 조직용 AI 하네스: 팀 전원이 쓰는 AI 개발 환경

원스토어 · 2026.01 ~ 현재 · 설계·구축·운영

**[Problem]**

- AI 도구를 쓰지 않는 팀원이 있어, 개인 선택에 맡기지 않고 팀 전원을 하나의 AI 개발 환경에 포함시켜야 했음
- 룰과 스킬이 계속 쌓이고 에이전트가 늘어나면서, 이를 효율적으로 관리할 구조가 필요해짐
- 업무 생산성을 높이려면 개발·운영·개선·점검 같은 일을 사람이 매번 계획하지 않아도 반복해서 끝까지 완료해야 했음

**[Solution]** 룰·스킬·에이전트를 하네스로 묶어 팀 전원의 공통 개발 환경으로 배포하고, 실행 방식을 스킬 워크플로에서 에이전트 그래프까지 발전시킴




하네스 실행 방식의 발전

**구성**




**배포와 사용**




하네스의 구성과 배포

- **업무 프로세스 자동화:** 팀 룰과 스킬, 스킬 조합으로 개발·리뷰·보고 같은 업무 흐름을 자동화
- **룰·훅으로 에이전트 제어:** 작업에 맞는 규칙만 읽게 해 토큰과 착각을 줄이고, 규약·테스트·아키텍처 판정은 훅과 스크립트로 검사해 같은 결과를 보장
- **에이전트 전문화:** 역할·분야·목적별로 에이전트를 나눠 환각을 줄이고 산출물 품질을 높임. planner는 계획만 세우고, 완료는 미리 정한 기준을 모두 채웠을 때만 인정
- **조직 전파:** Claude Code 플러그인 마켓플레이스로 같은 하네스를 조직에 전파·공유
- **개발 루프:** 기획 문서와 Jira 일감에서 시작해 테스트·구현·실기기 검증·리뷰·PR·완료 보고까지 자동화(뒤의 개발 루프 그림)
- **자가 개선:** 개발이 끝날 때 뽑아 둔 개선점과 매주 모은 사용 기록으로 하네스와 업무 자동화를 개선하고, 설정에 따라 자동으로 반영하거나 사용자에게 지시를 요청(뒤의 주간 개선 루프)
- **규모:** 에이전트 32종, 스킬 약 100개, 훅 25개

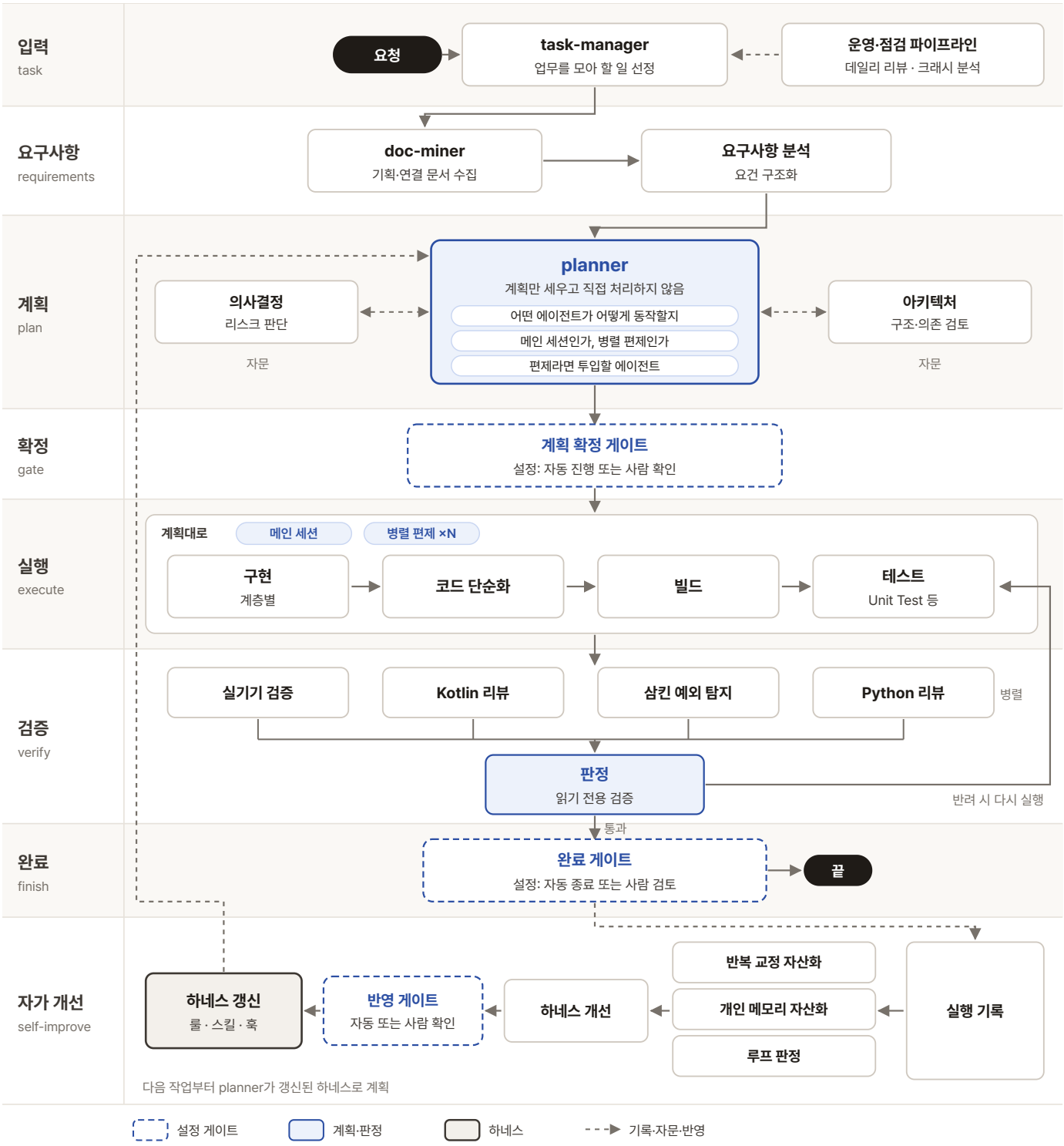
**[Result]**

- AI 도구를 쓰지 않던 팀원도 하네스로 개발하면서 AI 활용에 익숙해졌고, 지금은 팀원 전원이 이 하네스로 개발하고 사람은 지시·감독·검수를 맡음
- 도입 후 QA 검증 기간에 검출되는 이슈가 줄어듦
- 웹게임 플랫폼 단말 쪽을 팀이 약 3주 만에 개발하는 등 새 영역을 빠르게 도입
- 리뷰 강도를 단계로 나눠 PR 1건의 실측 토큰을 3.70M에서 1.49M으로 줄이면서 핵심 결함 2건은 그대로 잡음
- 사내 위키에 Claude Code 에이전트 가이드 12편을 쓰고, 하네스 구축 경험을 회사 발표로 공유

**기술:** Claude Code(플러그인·스킬·훅·에이전트), MCP, Jira·Confluence·Bitbucket·Teams 연동, 로컬 LLM Wiki, Kotest·MockK, Maestro

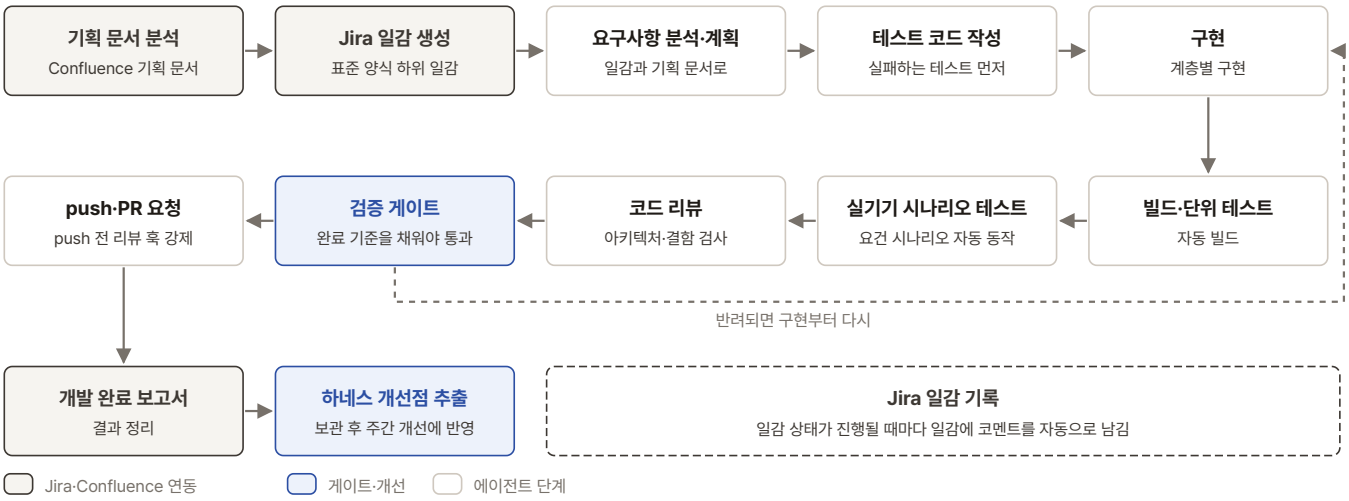
## 에이전트 그래프

조직용 AI 하네스 · 개발, 자가 개선, 운영·점검 파이프라인이 이어진 현재 구조



## Jira·Confluence 환경의 개발 루프

조직용 AI 하네스 · 기획 문서에서 시작해 완료 보고까지 이어지는 업무 흐름




- **기획에서 일감까지:** Confluence 기획 문서를 분석해 Jira 일감을 표준 양식으로 만들고, 일감과 기획 문서를 함께 읽어 요구사항을 분석하고 계획을 세움
- **테스트 먼저, 실기기까지:** 실패하는 테스트를 먼저 만든 뒤 구현하고, 자동 빌드와 단위 테스트 다음에 개발 요건 시나리오를 실제 단말에서 자동으로 돌려 확인
- **리뷰와 게이트:** 코드 리뷰와 검증 게이트를 통과할 때까지 반복하고, push 전에는 코드 리뷰를 훅으로 강제
- **진행이 Jira에 남음:** 일감 상태가 진행될 때마다 일감에 코멘트를 남기고, 끝나면 지정한 대상으로 push·PR을 요청하고 개발 완료 보고서를 작성
- **진척 확인:** 진행 상황이 일감에 남고 루프가 끝나면 Teams로 알려, 관리자는 에이전트를 통해 진척을 실시간으로 손쉽게 확인

### 단계별로 쓰는 하네스 장치

| 단계 | 커맨드 | 하네스 장치 |
|---|---|---|
| **기획·일감** | /jira-context | 기획 문서를 분석해 Jira 일감 생성, Jira·Confluence 요구사항을 기능 목록·우선순위·제약으로 구조화하는 스킬, 하위 일감을 표준 양식으로 만들고 서로 연결하는 스킬 |
| **계획** | /jira-start | 일감을 진행 상태로 바꾸고 착수 코멘트, 계층별 구현 순서와 파일 역할을 정하는 계획 스킬, 사람이 단 도메인 메모를 반영해 계획을 다시 다듬는 스킬 |
| **테스트·구현** | | 실패하는 테스트를 먼저 확인한 뒤 구현, 파일을 만들 때마다 아키텍처 규칙을 바로 검사 |
| **검증** | | 빌드·테스트·정적 분석·아키텍처·보안·인수 조건을 실행 결과로 확인하는 검증 루프, Maestro·mobile-mcp로 실기기 화면·로그·서버 응답을 함께 보는 UI 테스트 |
| **리뷰·게이트** | /jira-review | 일감 요구사항 기준 리뷰 결과를 일감에 코멘트, 아키텍처 정합성과 코드 결함을 같은 비중으로 보는 통합 코드 리뷰, 작업자와 분리된 읽기 전용 검증 에이전트, push 전 리뷰 훅 |
| **완료·보고** | /jira-complete<br>/jira-report | 지정한 대상으로 push·PR 요청과 일감 완료 처리, Jira 일감을 반영한 PR 메시지, 개발 완료 보고서를 Confluence에 발행하고 일감에 연결 |
| **감시** | /jira-watch | Jira 필터를 주기적으로 조회해 새로 들어온 일감을 Teams 카드로 알림 |

## 주간 개선 루프

조직용 AI 하네스 · 한 주 동안 쌓인 기록으로 보고서를 쓰고 하네스를 고치는 흐름

- **수집:** 매주 Claude의 세션 기록, 대화 내용, 스킬·훅 사용 기록, 모든 프로젝트 메모리와 개발 완료 때 뽑아 둔 개선점까지, 사용자에게서 얻을 수 있는 정보를 모음
- **주간보고 자동화:** 주간보고 스킬이 팀원마다 git, Jira, Confluence, 이메일 등 개인 채널과 Claude 사용 정보에서 한 주 업무를 뽑아 주간보고를 자동으로 작성
- **개선:** 같은 수집 결과에서 하네스 개선과 업무 자동화(스킬·훅·에이전트화)로 바꿀 수 있는 것을 정리하고, 설정에 따라 자동으로 개선하거나 사용자에게 지시를 요청

## 2. 기술 리드

### Clean Architecture 기반 앱 구조 설계와 팀 표준

원스토어 · 2022.07 ~ 현재 · 클라이언트개발팀 기술 리드(2023 ~)

**[Problem]**

- 15년 된 원스토어 앱에 MVC·일부 MVP 코드와 기술 부채가 쌓여 있음
- 서비스 중인 국내·글로벌 두 앱을 같은 구조로 옮기려면 팀이 함께 따를 기준이 필요

**[Solution]** Clean Architecture 개념을 도입해 앱의 계층과 의존 방향을 설계




의존은 바깥에서 Domain 쪽으로만 향합니다. Domain은 Android에 의존하지 않는 순수 Kotlin입니다.

| 의존성 역전 | 모듈 | 오류 |
| --- | --- | --- |
| Repository·DataSource 두 인터페이스 경계에만 | 계층별 모듈로 나누고 Hilt로 주입 | 예상된 실패는 Result, Data 계층에서 Domain 오류로 |

계층과 의존 방향

- **팀 합의:** 샘플 아키텍처 프로젝트로 먼저 설명하고, 계층별 모듈 구조와 개발 표준을 팀 합의로 확정
- **기반 기술:** Coroutine·Flow·Hilt·WorkManager 도입을 결정하고 Compose 전환을 팀에 전파
- **지키게 하는 장치:** 계층 규칙을 하네스 규칙과 훅으로 옮겨 사람과 AI가 같은 기준으로 코드를 씀. 코드 리뷰 가이드를 팀 규칙으로 만들고 대규모 리팩토링 병합은 직접 맡음
- **기술 부채:** 구조·레거시·의존성·테스트·시작 성능 과제를 정의해 팀 과제로 이끌고 진행을 관리
- **팀 학습:** MVVM·MVI, Clean Architecture, 헥사고날 아키텍처, DDD, 모듈화, 테스트를 주제로 팀 세션

**[Result]**

- 서비스 중인 국내·글로벌 두 앱이 같은 구조 표준으로 옮겨짐
- PR과 코드 리뷰가 팀 규칙으로 자리 잡음

**기술:** Kotlin, Clean Architecture, 멀티 모듈, MVVM·MVI, Hilt, Coroutine·Flow

### 항공 예약 파트 리더

AK아이에스 (제주항공) · 2019.06 ~ 2020.03

- 항공 예약 파트를 이끌며 예약 웹·앱 개발을 진행
- 고객사(제주항공) 담당자와 요건을 직접 협의·정리
- 개발 협력업체가 맡은 고객용·사내 웹과 모바일 앱의 코드 리뷰와 검수

## 3. 리팩토링

### 서비스 중인 국내·글로벌 앱의 전 계층 리팩토링

원스토어 · 2022.07 ~ 2025.09

**[Problem]**

- 로직이 화면 클래스와 화면별 Manager에 몰려 있고, 비동기는 AsyncTask, 화면은 XML 중심
- 많은 사용자가 쓰는 앱이라 서비스를 멈추지 않고 바꿔야 함

**[Solution]** 기능 개발과 함께 진행하며 국내 앱에서 설계한 구조를 글로벌 앱에도 적용

| 영역 | 이전 | 이후 |
|---|---|---|
| 화면 패턴 | MVC·일부 MVP | MVVM(단방향 데이터 흐름, Event-State 패턴), 지금은 MVI |
| 화면 | XML | Jetpack Compose |
| 비동기 | AsyncTask, 콜백 기반 로더 | Coroutine·Flow |
| 데이터 | 화면별 Manager, 데이터 매니저 싱글턴 | Repository, Hilt 주입 |
| 언어 | Java | Kotlin |

- **모듈 구성:** 계층별 모듈로 나누고, 공통 UI 컴포넌트와 스토어 화면, 설치 기능을 별도 모듈로 분리. flavor별 패키지·서명·설정으로 여러 브랜드 앱을 빌드하는 구조
- **기능 리팩토링:** 다운로드·설치 기능과 로그인 기능을 새 구조에 맞춰 직접 리팩토링

**[Result]**

- 서비스 중인 국내·글로벌 앱의 전체 계층이 새 구조로 옮겨짐

### 다운로드·설치 경로의 Service에서 WorkManager 전환

원스토어 · 2025.06 ~ 2025.09

**[Problem]**

- 앱 다운로드·통신사 업데이트·외부 연동·코어 업그레이드를 포그라운드 서비스 하나가 처리하는데, Android의 서비스 실행 제약은 버전마다 강해짐

**[Solution]**




전환 전후 구조

- 남은 기능까지 옮긴 뒤 서비스를 제거하고, 코어 업그레이드 모듈은 완료 통지를 옵저버로 바꿔 의존 방향을 바로잡음
- 배포 후 특정 단말에서 난 서비스 시작 시간 초과 예외를 추적해 알림 ID를 Worker별로 나누고 자동 업그레이드 Worker를 백그라운드로 바꿈

**[Result]**

- 국내 앱의 핵심 다운로드·설치·자동 업데이트 경로가 WorkManager로 옮겨지고 기존 서비스가 사라짐

**기술:** WorkManager, Foreground Service, Hilt, UseCase

## 4. 모바일 개발·운영

국내·글로벌 원스토어 앱에서 다운로드·설치·자동 업데이트, 전시, 회원·결제 연동을 맡고 있습니다. 아래는 판단 과정이 드러나는 사례입니다.

### 다운로드 APK 무결성 검증: 설치 시작 한 지점에서 대조

원스토어 · 2026.09 ~ 2026.10

**[Problem]**

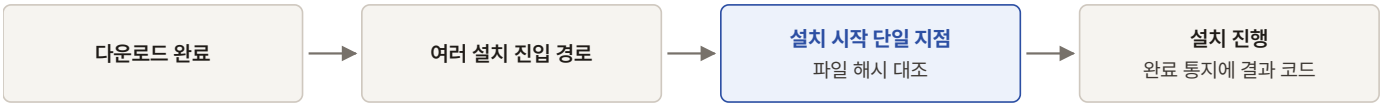
- 보안 점검에서 원스토어 앱이 내려받은 APK의 위변조를 확인하라는 지적을 받음
- 서버의 HTTPS 전환은 회선 과금 구조 때문에 어려움

**[Solution]**

**방향 결정**




**대조 위치**




모든 설치 경로가 한 지점의 대조를 지나게 해 경로마다 검사를 따로 두지 않았습니다.
대조 도중 앱이 종료되면 검증 전 파일이 설치되던 빈틈도 막았습니다.

방향 결정과 대조 위치

- 서버에 무엇을 더 요청할지 정하기 전에 기존 응답부터 조사해, 새로 받는 값을 파일 해시 하나로 줄임
- 서버 방침이 정해진 뒤에는 불일치 시 설치를 막지 않고 완료 통지에 결과 코드를 담도록 맞춤

**[Result]**

- 모든 설치 경로가 같은 대조를 지나게 됨

**확인 범위:** QA 실기기에서 대조 일치와 설치 진입을 확인했습니다. 변조 파일 차단은 단말에서 재현하지 못해 단위 테스트로만 확인했습니다. AAB 분할 파일은 서버에 파일별 해시가 생기기 전까지 대상이 아닙니다.

### 세션 만료 복구

원스토어 · 2026.09 ~ 2026.10

- **[Problem]** 앱을 오래 둔 뒤 돌아오면 카드 화면이 흰 화면이 됨
- **[Solution]** 수정 전 빌드에서 서버 만료 응답을 흉내 내 증상을 재현한 뒤 고침. 세션 연장까지 실패하면 재로그인이 필요한 상태로 바꾸고, 예외를 로그로만 남기던 화면에 기존 에러 화면(새로고침)을 띄움. 웹게임 서버 호출도 Data 계층으로 옮겨 같은 처리를 적용. 서버의 새 암호화 규격에 맞춰 요청·필드별 IV를 적용하고 서버 회신과 대조해 일치를 확인
- **[Result]** 세션이 끊겨도 빈 화면 대신 재로그인이나 새로고침으로 이어짐
- **확인 범위:** 재로그인은 실기기에서 정상 경로를 확인했고, 서버 만료 상태에서의 복구 경로는 확인이 남아 있습니다.

### 설치 비호환 단말 관리 (국내·글로벌)

원스토어 · 2025.01 ~ 2025.04

- **[Problem]** 사전 승인이 안 되는 단말, AAB를 지원하지 않는 단말, 설치 확인 화면이 없는 단말에서 앱 설치가 실패
- **[Solution]** 서버 연동 API·로컬 DB·서버 동기화로 비호환 단말을 관리. 사전 승인이 안 되는 단말은 사전 승인을 건너뛰고, AAB 미지원 단말은 APK로 요청. 신뢰된 제조사 단말은 조회를 건너뛰고 동기화 시점을 앱 시작으로 옮김
- **[Result]** 같은 관리 로직을 국내·글로벌 앱의 다운로드·설치에 연결

# 5. SDK 개발

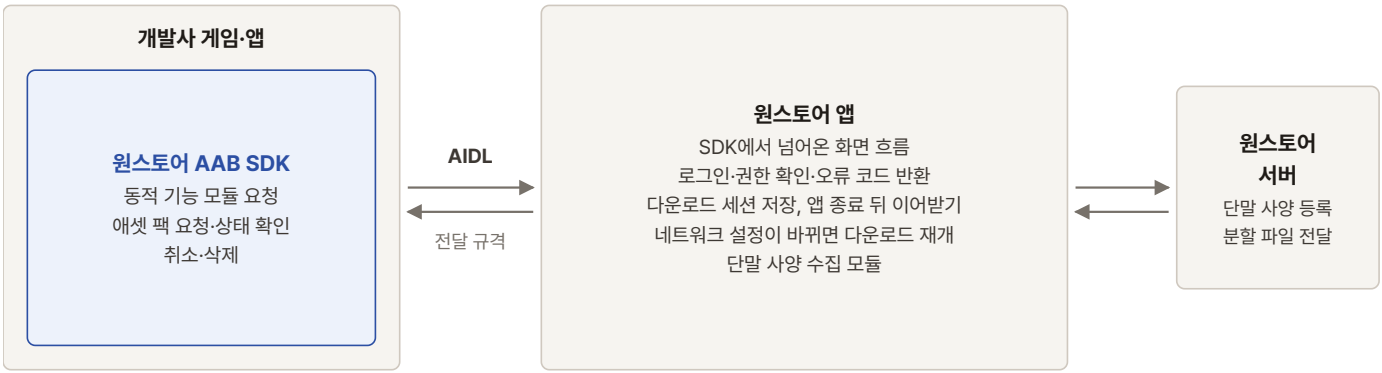
## AAB Dynamic Delivery SDK

원스토어 · 2020.10 ~ 2021.10

**[Problem]**

- 원스토어는 AAB(Android App Bundle)를 지원하지 않아, 게임·앱 개발사가 동적 기능 모듈과 애셋 팩을 원스토어에서 필요할 때 내려받을 방법이 없음

**[Solution]**




SDK, 원스토어 앱, 서버 사이의 역할

- **역설계:** Google Play Core를 리버스 엔지니어링해 구조를 역설계하고, AAB protobuf 구조·Feature 모듈 매니페스트·bundletool을 조사
- **채택 판단:** 역설계 결과를 바탕으로 그대로 가져올 부분과 원스토어 설계에 맞게 바꿀 부분을 가르고, 내부 상세 동작을 분석해 SDK로 구현
- **경계 설계:** SDK의 인터페이스와 기능을 정의하고, SDK의 요청을 원스토어 앱이 받아 처리하도록 둘 사이의 AIDL 인터페이스와 전달 데이터 규격을 설계·구현
- **원스토어 앱 쪽:** 다운로드 세션을 저장해 원스토어 앱 종료 뒤 생기던 다운로드 오류를 해결. 라이브러리 내부 Lock으로 생긴 ANR 수정. 분할 전달에 필요한 단말 사양 수집 모듈 추가
- **개발사 지원:** 검증용 샘플 앱과 SDK 개발 가이드 작성

**[Result]**

- 원스토어 앱에 탑재되는 형태로 일부 기능을 상용 적용

**기술:** Android App Bundle, bundletool, AIDL, protobuf, ProGuard

## 그 밖의 SDK·라이브러리·도구

| 기간 | 이름 | 내용 |
|---|---|---|
| — | 원스토어용 bundletool 라이브러리 (원스토어) | 원스토어 앱 빌드 서버가 불러 쓰는(import) 원스토어용 bundletool을 JAR 라이브러리로 개발 |
| 2021.11 ~ 2022.03 | APK·AAB 리패키징·재서명 도구 (원스토어) | 글로벌 스토어 개발사 앱을 다시 패키징·서명하는 도구를 Kotlin 실행 가능한 JAR로 개발. bundletool·apksig로 서명 체계 검증, 여러 서명 키 처리 |
| 2014.07 ~ 2014.10 | SK Physics Android SDK (이노에이스) | Android View와 Box2D 물리엔진을 연동하는 SDK 공동 개발. 앱 개발자용 ForceLayout 사용 구조 설계, 물리 스레드에서 계산한 Matrix로 View 화면 갱신 |

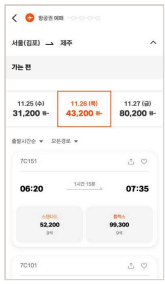
# 이전 프로젝트 화면

## 제주항공 앱·웹 | AK아이에스 | 2018.07 ~ 2020.03

서비스 중인 고객용 iOS 앱을 맡아 회원, 항공권 예매, 모바일 탑승권, 웹–앱 네이티브 브릿지를 개선·고도화하고, 예약 웹 화면을 개발했습니다.



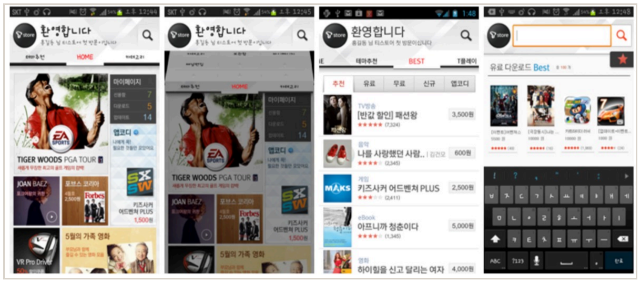

제주항공 모바일 웹 홈 (2026년 화면)




제주항공 항공권 예매: 운항편·운임 선택 (2026년 화면, 재직 당시와 흐름 유사)

## 앱마켓·런처와 그 밖의 프로젝트 | 이노에이스 | 2010.01 ~ 2017.10

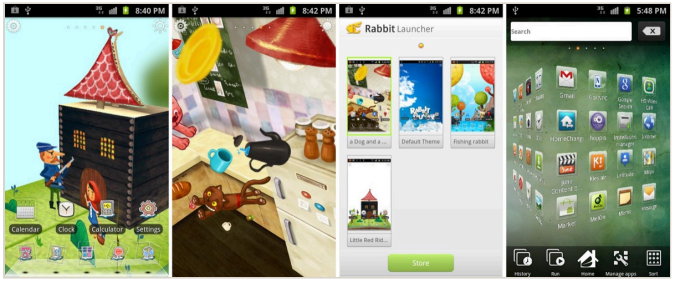
티스토어·원스토어 앱의 UI와 비즈니스 로직, 제조사 홈과 바꾸는 T간편모드, SKT 선탑재 런처와 글로벌 런처를 개발했고, KTOON·GameCast와 HTML5 웹앱 저작 도구(다빈치)를 개발했습니다.




티스토어: 홈, 베스트 목록, 검색 화면



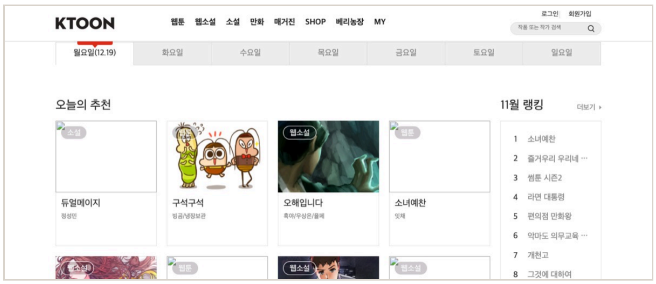

T간편모드: 홈 모드 전환, 키패드형 홈, 화면·소리 설정, 간편 메뉴



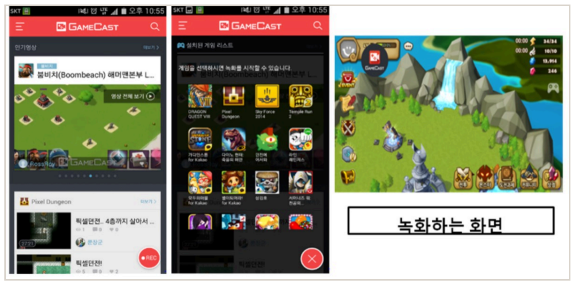

Rabbit Launcher: 3D 테마 홈, 테마 선택, 앱 서랍



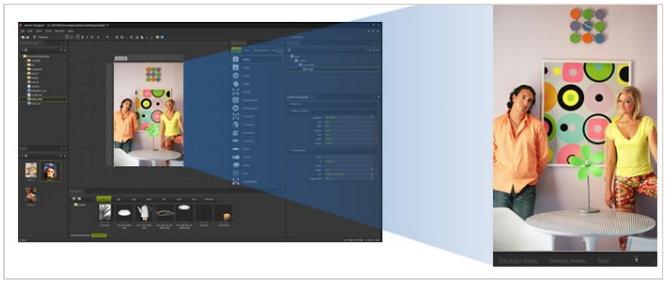
폰꾸미기: 테마 상점과 테마 꾸미기, CUKI 런처 메뉴



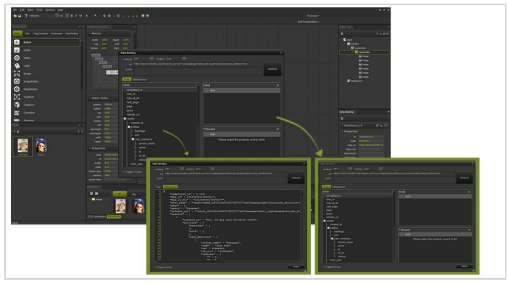

KTOON: 2016년 12월 웹 메인 화면




GameCast: 인기 영상, 녹화할 게임 목록, 게임 녹화 화면



다빈치 Studio: WYSIWYG 편집 화면과 에뮬레이터 미리보기



다빈치 Studio: 컴포넌트 배치와 데이터 바인딩 설정